

第7章 进程管理





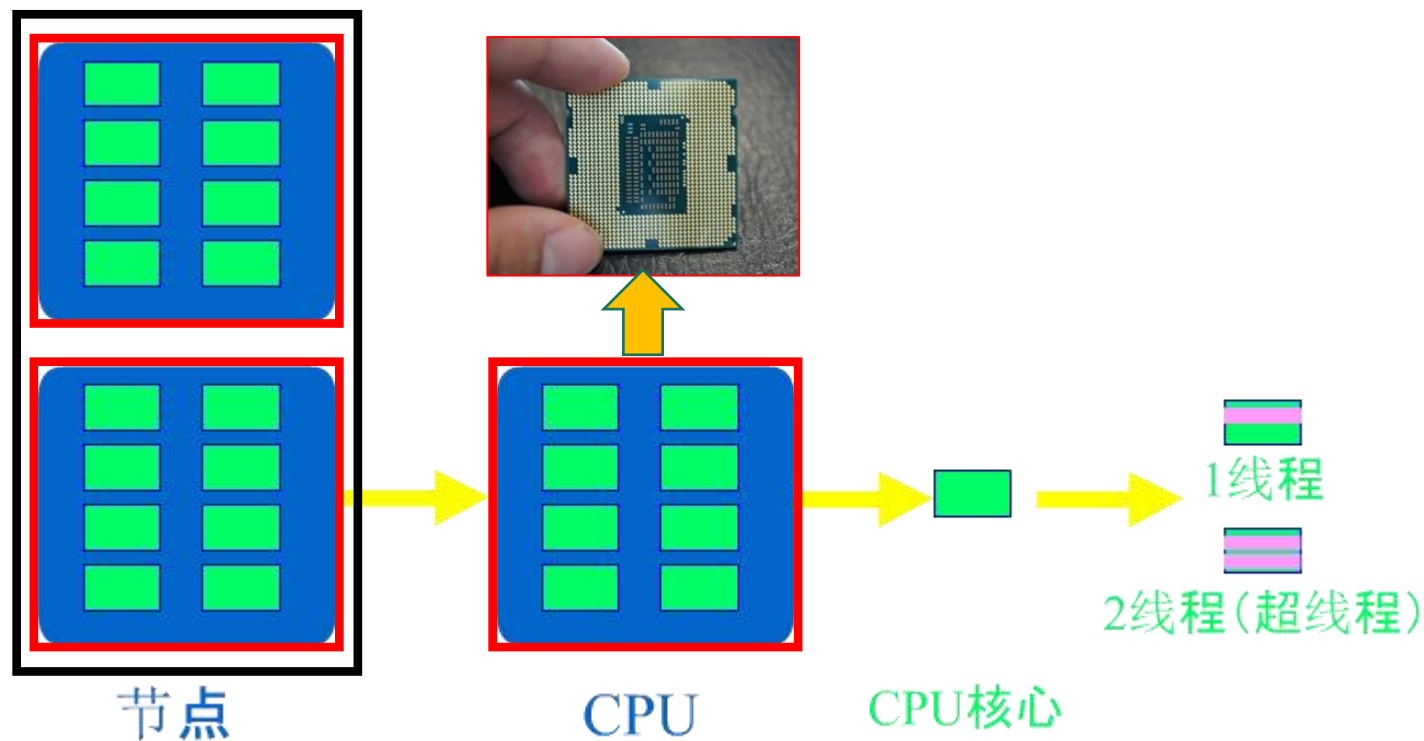
节点、cpu、cpu核、进程、线程、 运行程序



第7章 进程管理

节点、cpu、cpu核

- 1) 1个CPU有1或多个CPU核心；
- 2) 一个节点也可以有1或多个物理CPU。

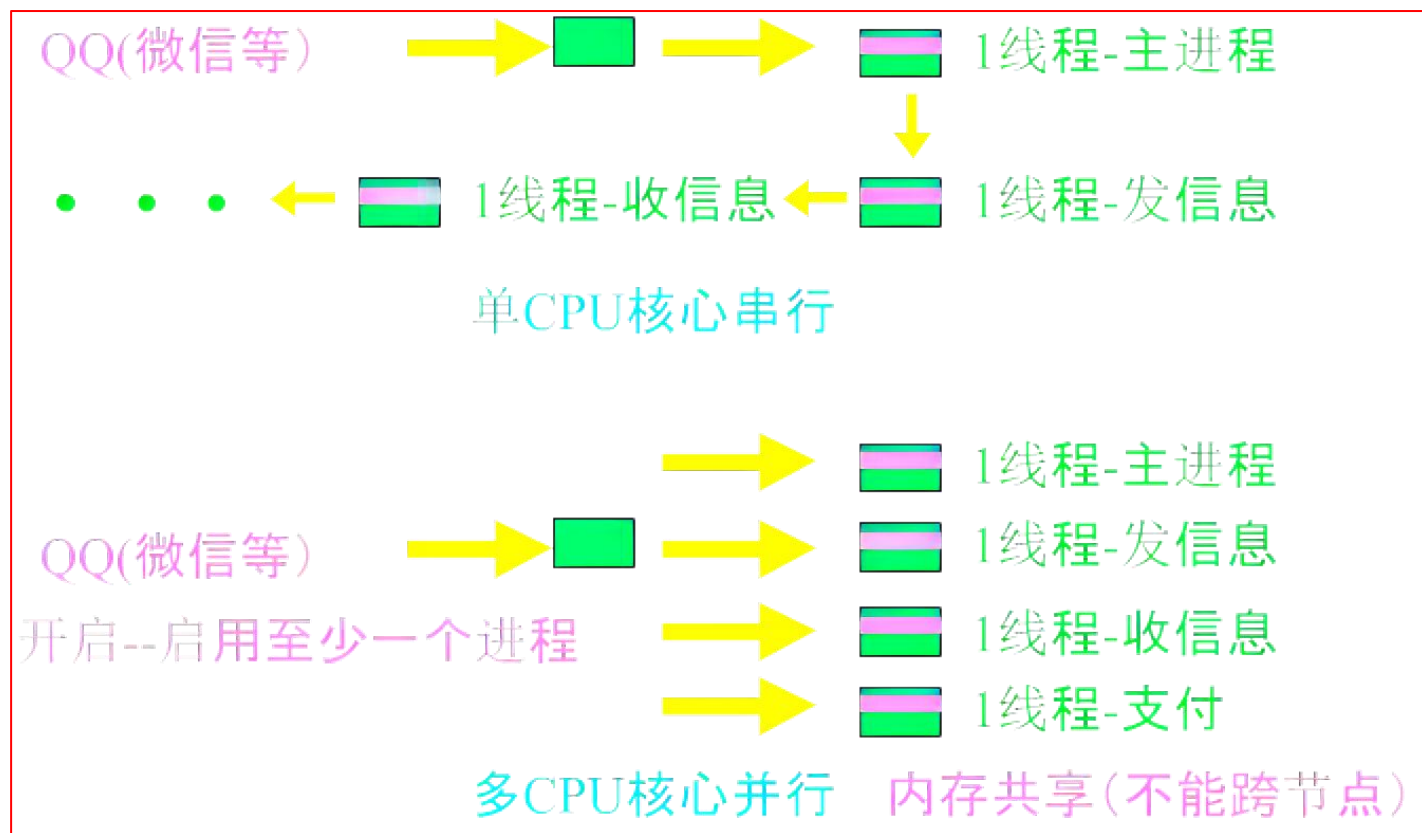




第7章 进程管理

进程、线程

- 1) **进程**是操作系统进行资源（包括cpu、内存、磁盘IO等）分配的最小单位；
- 2) **线程**是cpu调度和分配的基本单位。





为何要掌握进程管理？

- 1) Linux是一种多用户/多任务系统，多任务意味着可以同时运行多个程序；
- 2) 监视系统的使用情况；
- 3) 在必要时启动、暂停、停止或者杀死进程。

```
top - 11:51:37 up 57 min, 1 user, load average: 0.00, 0.00, 0.00
Tasks: 110 total, 1 running, 109 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.2 sy, 0.0 ni, 96.2 id, 0.0 wa, 1.5 hi, 2.2 si, 0.0 st
MiB Mem : 1976.8 total, 1577.5 free, 172.7 used, 226.6 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used. 1646.2 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
2742	root	20	0	0	0	0	I	0.3	0.0	0:00.96	kworker/0:2-events
2786	root	20	0	18320	4328	3660	R	0.3	0.2	0:00.08	top
1	root	20	0	172372	15396	10420	S	0.0	0.8	0:01.38	systemd



Linux进程概述

- 程序和进程

- 程序：静态概念，代表一个可执行的二进制文件。

- 例如：/bin/date, /bin/bash, /usr/sbin/sshd等都是Linux下可执行的二进制文件。

- 进程：动态概念，代表程序运行的过程。

- 进程有其生命周期及运行状态。

通俗的话来讲,进程就是一个程序的一次执行过程。进程是一个动态实体,程序作为软件资料可以长期保存,而进程则有生命期,它由创建而诞生,因调度而执行,因得不到资源而暂停执行,因撤销而消亡。



问：进程和程序的一个本质区别是（ ）

- A. 前者分时使用CPU，后者独占CPU
- B. 前者存储在内存，后者存储在外存
- C. 前者在一个文件中，后者在多个文件中
- D. 前者为动态的，后者为静态的



Linux进程概述

• 进程的状态

• 运行 (running) 态:

- 进程占有处理器正在运行。

• 就绪 (ready) 态:

- 进程具备运行条件，等待系统分配处理器以便运行。

• 等待 (wait) 态:

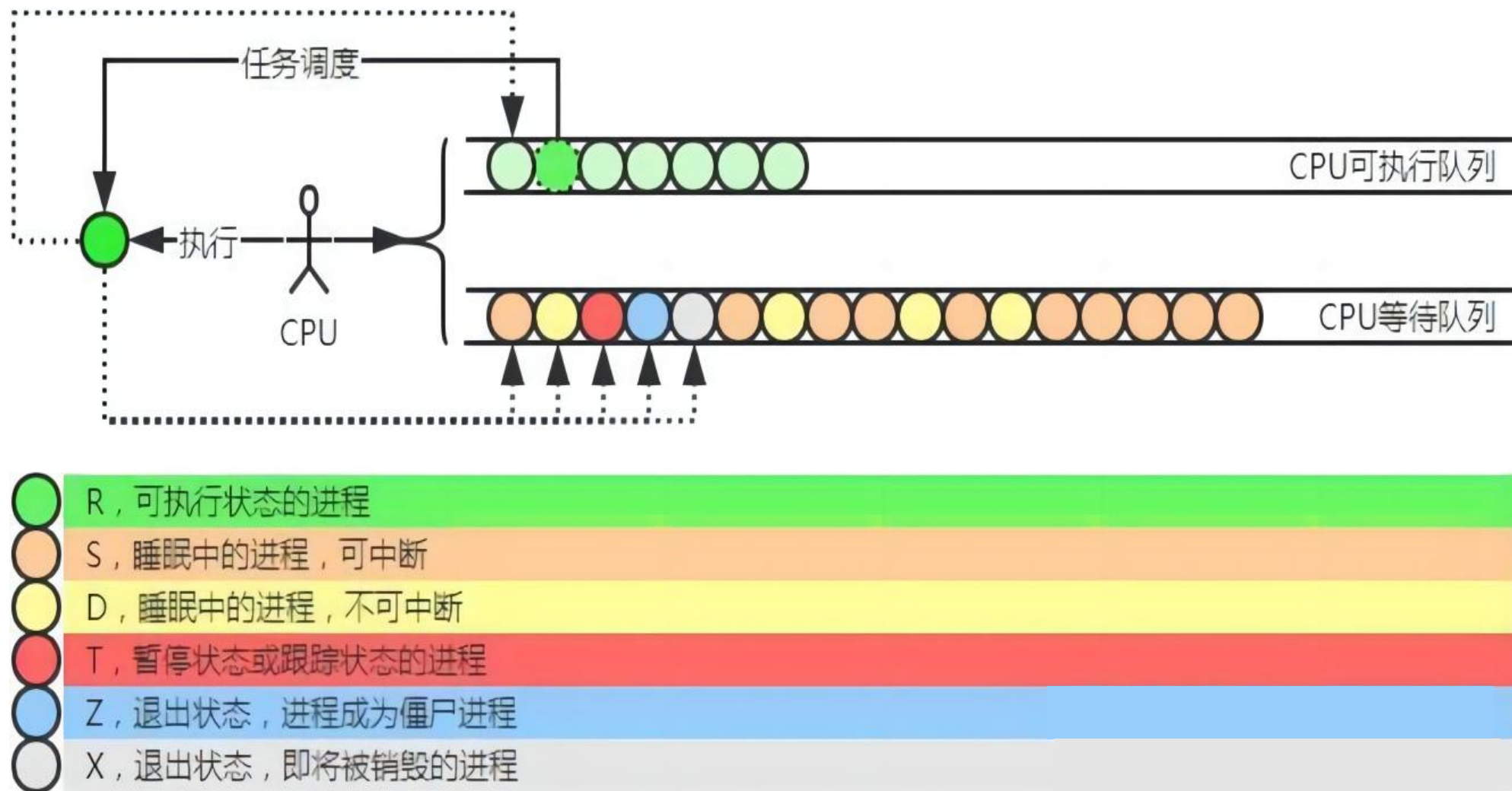
- 又称为阻塞 (blocked) 态或睡眠 (sleep) 态，指进程不具备运行条件，正在等待某个事件的完成。即使得到处理器也不能运行的状态。

```
1  /* 内核代码
2  * The task state array is a strange "bitmap" of
3  * reasons to sleep. Thus "running" is zero, and
4  * you can test for combinations of others with
5  * simple bit tests.
6  */
7  //保存Linux状态的指针数组
8  static const char * const task_state_array[] = {
9  "R (running)", /* 0 */
10 "S (sleeping)", /* 1 */
11 "D (disk sleep)", /* 2 */
12 "T (stopped)", /* 4 */
13 "t (tracing stop)", /* 8 */
14 "X (dead)", /* 16 */
15 "Z (zombie)", /* 32 */
16 };
```



Linux进程概述

linux系统中，将各种状态进行了重新组织：



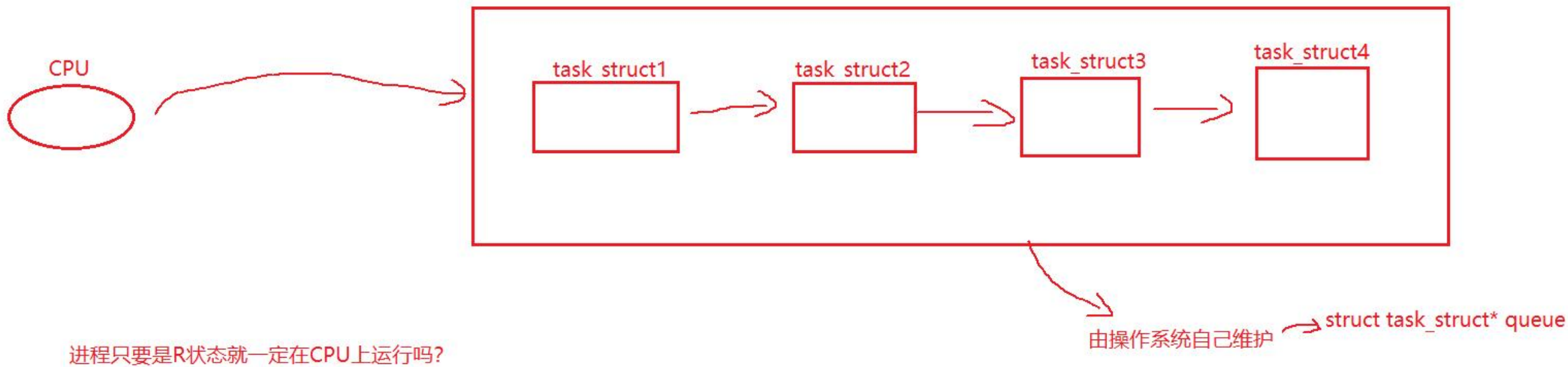


Linux进程概述

1) **R状态（运行态）**：正在运行的，以及拿到时间片就能运行的(RUNNING)。

```
[hjt@VM-12-5-centos ~]$ ps aux | head -n 10 | grep state | grep -v grep
PPID    PID    PGID    SID    TTY      TPG  D  STAT    UID     TIME    COMMAND
8909    9549    9549    8909    pts/0    95   9  R+      0        0:47    ./state
```

运行队列





Linux进程概述

1) R状态（运行态）：正在运行的，以及拿到时间片就能运行的(RUNNING)。

前台进程：前台进程会影响命令行交互，进程状态前面有个**+**号，代表前台进程，前台进程一启动，执行命令没有效果，可以被键盘Ctrl+c终止。

```
#####
PPID  PID  PGID  SID TTY      TPGID  STAT  UID   TIME COMMAND
17378 23672 23672 17378 pts/0    23672 R+    1001   0:01 ./myproc
#####
PPID  PID  PGID  SID TTY      TPGID  STAT  UID   TIME COMMAND
17378 23672 23672 17378 pts/0    23672 R+    1001   0:02 ./myproc 前台进程
#####
PPID  PID  PGID  SID TTY      TPGID  STAT  UID   TIME COMMAND
17378 23672 23672 17378 pts/0    23672 R+    1001   0:03 ./myproc
```

后台进程：后台进程不影响命令行交互，执行程序./myproc **&** 进程就在后台运行，那么输入各自命令也能正常执行，进程一直在后台运行。（Ctrl+c无法终止，如果要kill该进程，可以执行kill -9 id号）

```
#####
PPID  PID  PGID  SID TTY      TPGID  STAT  UID   TIME COMMAND
17378 22880 22880 17378 pts/0    17378 R    1001   0:04 ./myproc
#####
PPID  PID  PGID  SID TTY      TPGID  STAT  UID   TIME COMMAND
17378 22880 22880 17378 pts/0    17378 R    1001   0:05 ./myproc
```



Linux进程概述

kill -9 pid # 杀掉进程

2) S状态(休眠): 可中断休眠, 本质上也是一种阻塞状态, 等待某种资源就绪。

```
#include <stdio.h>
#include <unistd.h>
int main()
{
while(1)
{
printf("我在运行吗???\\n");
sleep(1);
}
return 0;
}
```

PPID	PID	PGID	SID	TTY	TPGID	STAT	UID	TIME
1	9832	9832	9832	?	-1	SsL	1003	0:00
24354	25177	25177	24354	pts/1	25177	S+	1012	0:00

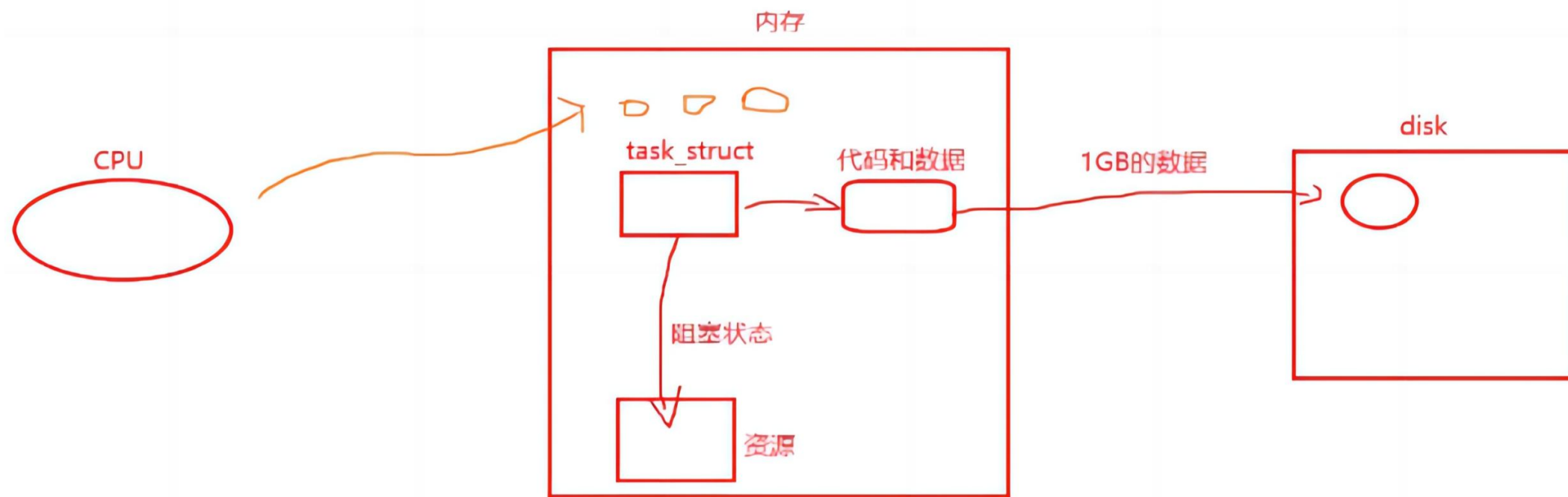
为什么是sleep?

注释掉printf函数向外设打印字符, 看看此时进程处于什么状态?



Linux进程概述

3) D状态(disk sleep)：深度休眠状态(不可中断休眠状态)，平常我们几乎看不到这种休眠状态。





Linux进程概述

4) T状态(暂停): 进程被暂停, 需要通过其它进程的信号才能被唤醒。

通过发送 **SIGSTOP** 信号给进程来停止 (T) 进程。这个被暂停的进程可以通过发送 **SIGCONT** 信号让进程继续运行。

```
1 | kill -19 pid # 停止pid进程
```

```
1 | kill -18 pid # 恢复停止进程的信息, 使其继续执行
```



Linux进程概述

5) Z状态(僵尸状态)：进程处于僵尸状态，已经不用上处理机运行的状态。

当进程结束的时候，需要将进程的结束信息（是否完成任务等）告诉给父进程或者操作系统，操作系统会为其维护一个称为僵尸态的状态。

```
drwxr-xr-x 9 root root 4096 Nov 10 22:55 code
[root@VM-12-5-centos ~]# kill -9 13723
[root@VM-12-5-centos ~]# ps ajx|head -1&&ps ajx|grep state|grep -v grep
  PPID    PID  PGID   SID TTY      TPGID STAT   UID    TIME COMMAND
   8909  13722 13722   8909 pts/0    13722 S+      0      0:00 ./state
13722  13723 13722   8909 pts/0    13722 Z+      0      0:00 [state] <defunct>
[root@VM-12-5-centos ~]#
```



5) Z状态(僵尸状态)：进程处于僵尸状态，已经不用上处理机运行的状态。

僵尸进程的危害：

- 1)** 可父进程如果一直不读取，那子进程就一直处于Z状态？是的！
- 2)** 维护退出状态本身就是要用数据维护，也属于进程基本信息，所以保存在task_struct (PCB) 中，换句话说，Z状态一直不退出，PCB一直都要维护？是的！
- 3)** 那一个父进程创建了很多子进程，就是不回收，是不是就会造成内存资源的浪费？



Linux进程概述

思考题：父进程如果提前退出，那么子进程如何处理？

“孤儿进程”？

关于子进程和父进程的说法，下面哪一个是正确的？（）

- A. 一个父进程可以创建若干个子进程，一个子进程可以从属于若干个父进程
- B. 父进程被撤销时，其所有子进程也被相应撤销
- C. 子进程被撤销时，其从属的父进程也被撤销
- D. 一个进程可以没有父进程或子进程



Linux进程概述

- 常见的进程

- 僵尸进程：

- 一个进程使用fork创建子进程，如果子进程退出，而父进程并没有调用wait或waitpid获取子进程的状态信息，那么子进程的进程描述符仍然保存在系统中。子进程就没有父进程来管理，因此就变成了僵尸进程。

- 交互进程：

- 通常是由终端启动的进程。交互进程既可以在前台运行，也可以在后台运行。

- 批量处理进程：

- 是一个进程序列，通常和终端没什么联系，通常以脚本或命令文件的形式存在。

- 守护进程（daemon）：

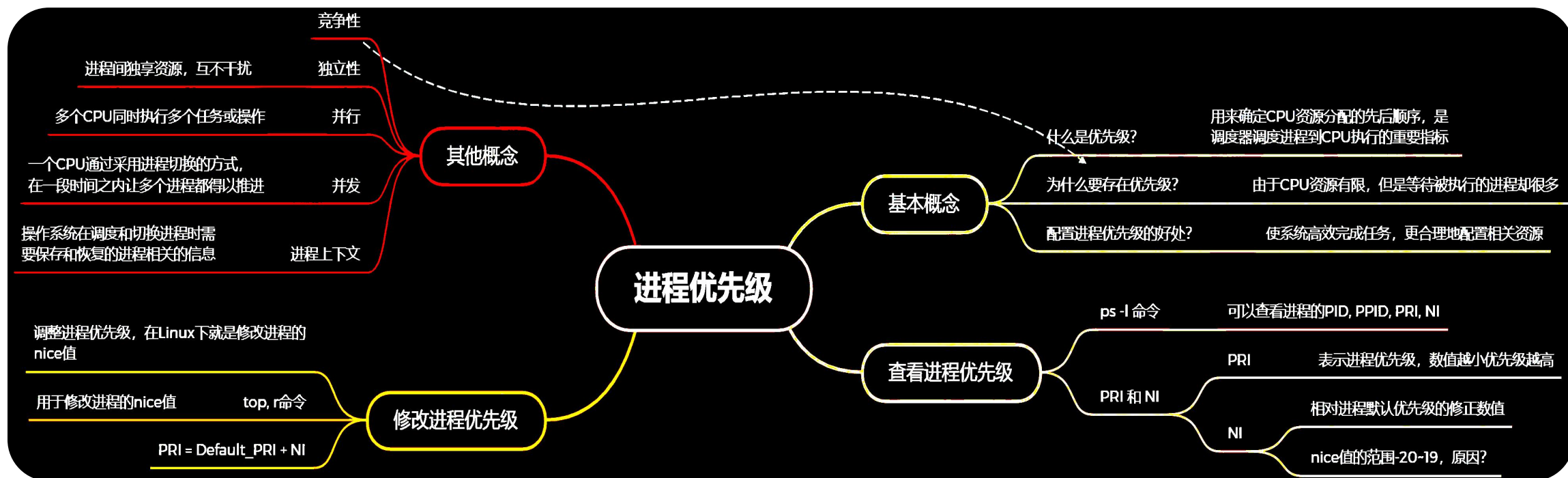
- 是一类在后台运行的特殊进程，用于执行特定的系统任务。通常在系统引导的时候启动，并且一直运行直到系统关闭。常见的守护进程包括网络服务进程、系统监控进程等。



Linux进程概述

• 进程优先级

- 优先级代表对 **CPU** 的优先使用级别。
- 优先级概念的具体含义与调度算法有关。
- 进程调度算法以进程的优先级为基础。





Linux进程概述

- `niceness`值为负时，友善度低，表示进程具备高优先级

```
zp@lab:~$ nice vi&
[1] 22951
zp@lab:~$ ps -l
```

F	S	UID	PID	PPID	C	PRI	NI	ADDR	SZ	WCHAN	TTY	TIME	CMD
0	S	1000	13442	13441	0	80	0	-	4984	do_wai	pts/0	00:00:00	bash
0	T	1000	22951	13442	0	90	10	-	8275	do_sig	pts/0	00:00:00	vi
0	R	1000	22952	13442	0	80	0	-	5009	-	pts/0	00:00:00	ps

```
[1]+  已停止                  nice vi
```

```
zp@lab:~$ sudo nice -n -15 vi&
[2] 8260
zp@lab:~$ sudo ps -l
```

F	S	UID	PID	PPID	C	PRI	NI	ADDR	SZ	WCHAN	TTY	TIME	CMD
4	T	0	8239	7825	0	80	0	-	5061	do_sig	pts/2	00:00:00	sudo
4	T	0	8260	7825	0	80	0	-	5151	do_sig	pts/2	00:00:00	sudo
4	T	0	8261	8260	2	65	-15	-	8275	do_sig	pts/2	00:00:00	vi

PRI and NI

进程的PRI为: $PRI = \text{Default_PRI} + NI$



Linux进程概述

```
nice -n 數字 command
```

示例：

```
nice -n -10 vi
```

设置进程的优先级的修正值为-10

```
nice -19 vi
```

此时-不标识负数，而表示参数指令，以19优先级来启动。



Linux进程概述

renice [优先等级][**-g** ...][**-p** ...][**-u** ...]

-g 使用程序群组名称，修改所有隶属于该程序群组的程序的优先权。

-p 改变该程序的优先权等级，此参数为预设值。

-u 指定用户名称，修改所有隶属于该用户的程序的优先权。

```
zp@lab:~$ ps -l
F S  UID  PID  PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
0 S  1000  25151 25045 0  80   0  -  4887 do_wai pts/1    00:00:00 bash
0 D  1000  37475 25151 99  80   0  -  7683 generi pts/1    00:00:07 vi
0 R  1000  37483 25151 0  80   0  -  5009 -      pts/1    00:00:00 ps
zp@lab:~$
zp@lab:~$ renice -8 -p 37475
renice: 设置 37475 的优先级失败(process ID): 权限不够

[1]+  已停止                  vi
zp@lab:~$ sudo renice -8 -p 37475
[sudo] zp 的密码:
37475 (process ID) 旧优先级为 0，新优先级为 -8
zp@lab:~$
zp@lab:~$ ps -l
F S  UID  PID  PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
0 S  1000  25151 25045 0  80   0  -  4887 do_wai pts/1    00:00:00 bash
0 T  1000  37475 25151 9  72  -8  -  8224 do_sig pts/1    00:00:09 vi
0 R  1000  37518 25151 0  80   0  -  5009 -      pts/1    00:00:00 ps
```



第7章 进程管理

1. Linux进程概述
2. 进程状态监测
3. 进程状态控制
4. 进程启动与作业控制



查看当前进程状态的命令ps

ps 的英文全称是 **process status**，意思是进程状态。

查看**当前时刻的**系统中运行的进程信息，如**进程 ID (PID)**、**进程状态**、**CPU 使用情况**、**内存占用**、**运行时间**等。（是固定的，当前的，静态的）

- **-e** : (every) 显示所有进程，而不仅仅是当前用户的进程
- **-u** : (user) 显示面向用户的进程的详细信息，如进程所有者、CPU 使用率、内存使用率等
- **-f** : (full-format) 以全格式的列表显示进程的信息，包括进程之间的层级关系
- **-p** : (pid) 查找具有指定进程 ID 的进程
- **-C** : (Command) 查找具有指定命令名称的进程
- **-aux** : 以详细格式显示所有正在运行的进程，其中 **x** 指的是在没有控制终端的情况下列出进程，这些显示的主要是启动并运行在后台的进程



静态监控：查看当前进程状态的命令ps

```
john@lab:~$ ps
```

PID	TTY	TIME	CMD
2868	pts/0	00:00:00	bash
2919	pts/0	00:00:00	ps

```
john@lab:~$ ps -l
```

F	S	UID	PID	PPID	C	PRI	NI	ADDR	SZ	WCHAN	TTY	TIME	CMD
0	S	1000	2868	2867	0	80	0	-	4803	do_wai	pts/0	00:00:00	bash
0	R	1000	2927	2868	0	80	0	-	5002	-	pts/0	00:00:00	ps

```
john@lab:~$ ps -u john -l
```

F	S	UID	PID	PPID	C	PRI	NI	ADDR	SZ	WCHAN	TTY	TIME	CMD
4	S	1000	2106	1	0	80	0	-	4730	ep_pol	?	00:00:00	systemd
5	S	1000	2107	2106	0	80	0	-	42370	-	?	00:00:00	(sd-pam)
0	S	1000	2119	2106	0	69	-11	-	223378	poll_s	?	00:00:00	pulseaudio
1	S	1000	2122	1	0	80	0	-	62236	-	?	00:00:00	gnome-keyring-



第7章 进程管理

F : 代表这个进程标志，说明这个程序的权限。

常见号码有：

1 : 表示此子进程仅进行复制，而没有进行实际操作。

4 : 表示此程序的权限为root

S : 代表这个程序的状态（STAT），主要的状态有：

R : 表示这个程序正在运行中；

S : 表示该程序目前正在睡眠状态，但可以被唤醒。

D : 表示不可被唤醒的睡眠状态，通常可能是在等待 I/O 操作的情况。

T : 停止状态，可能是在工作控制（后台暂停）或traced状态。

Z : 僵尸状态，程序已经终止但却无法被移除至记忆体外。

+ : 位于前台的进程。

C : 代表CPU使用率，单位为百分比。

ADDR/SZ/WCHAN : 都与内存有关。

ADDR : 指出该程序指出该程序在内存的哪个部分，如果是 running 的程序，一般会显示 - 。

SZ : 代表此程序用掉多少内存。

WCHAN : 表示目前程序是否在运行中，若为 - 表示正在运行。

TTY : 登入者的终端机位置，若为远程终端登入则使用动态终端界面（pts/n）。

Time : 使用掉的CPU时间，即此程序实际花费CPU运作的时间。



【实例】查找指定进程信息。

`ps` 可以通过管道符 `|` 和其他命令结合使用。

例如：将 `ps` 命令的输出通过管道交给 `grep` 命令进行匹配，找到我们想要的进程，起到一个过滤输出显示结果的作用。

```
john@lab:~$ ps -ef|grep python
root          978          1  0 20:17 ?                00:00:00 /usr/bin/python3 /usr/bin/networkd-dispatcher --run-startup-triggers
root         1199          1  0 20:17 ?                00:00:00 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-signal
john         4621       2868  0 20:54 pts/0            00:00:00 grep --color=auto python
```

```
john@lab:~$ ps auxf|grep python
root          978  0.0  0.6 47248 19916 ?        Ss   20:17   0:00 /usr/bin/python3 /usr/bin/networkd-dispatcher --run-startup-triggers
root         1199  0.0  0.7 126264 22896 ?        Ssl  20:17   0:00 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-signal
john         4717  0.0  0.0 17540   916 pts/0    S+   21:01   0:00 grep --color=auto python
```



动态监控：持续监测进程运行状态的命令top

```
top - 21:09:22 up 52 min, 2 users, load average: 0.12, 0.03, 0.08
任务: 275 total, 1 running, 274 sleeping, 0 stopped, 0 zombie
%Cpu(s): 1.0 us, 0.7 sy, 0.0 ni, 98.3 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 2977.7 total, 884.8 free, 832.8 used, 1260.1 buff/cache
MiB Swap: 947.2 total, 947.2 free, 0.0 used. 1967.8 avail Mem
```

进程	USER	PR	NI	VIRT	RES	SHR	%CPU	%MEM	TIME+	COMMAND
205	root	0	-20	0	0	0 I	0.3	0.0	0:05.30	kworker/0+
2146	john	20	0	248780	50876	33868 S	0.3	1.7	0:01.46	Xorg
2352	john	20	0	2370072	247712	103556 S	0.3	8.1	0:08.56	gnome-she+
2556	john	20	0	146400	27460	24332 S	0.3	0.9	0:03.35	vmtoolsd
2867	john	20	0	13272	6456	5216 S	0.3	0.2	0:00.75	sshd
4994	john	20	0	20596	4128	3376 R	0.3	0.1	0:00.05	top
1	root	20	0	168688	12668	7904 S	0.0	0.4	0:02.44	systemd
2	root	20	0	0	0	0 S	0.0	0.0	0:00.00	kthreadd

第一行，与 uptime 命令的执行结果相同。

第二行，当前运行的各类不同状态任务（进程）的数量。

第三行，CPU 状态信息。

第四行，内存状态。

第五行，swap 交换分区信息。

第六行，空行。

第七行，各进程（任务）的状态监控。

进程 — 进程的 PID。

USER — 进程所有者。

PR — 进程优先级。

NI — nice 值。负值表示高优先级，正值表示低优先级。

VIRT — 进程使用的虚拟内存总量，单位 kb。VIRT=SWAP+RES。

RES — 进程使用的、未被换出的物理内存大小，单位 kb。RES=CODE+DATA。

SHR — 共享内存大小，单位 kb。

进程状态 — D=不可中断的睡眠状态；R=运行；S=睡眠；T=跟踪/停止；Z=僵尸进程。

%CPU — 上次更新到现在的 CPU 时间占用百分比。

%MEM — 进程使用的物理内存百分比。

TIME+ — 进程使用的 CPU 时间总计，单位 1/100 秒。

COMMAND — 进程名称（命令名/命令行）。



【实例】使用top命令监控某个进程。

```
john@lab:~$ ps aux|grep sshd
root      1267   0.0   0.2  12028  6816 ?        Ss   20:17   0:00 /usr/sbin/sshd
-D
root      2753   0.0   0.2  12960  8164 ?        Ss   20:20   0:00 sshd: john [pri
v]
john      2867   0.0   0.2  13272  6456 ?        S    20:20   0:01 sshd: john@pts/
0
john      5117   0.0   0.0  17672   920 pts/0    S+   21:14   0:00 grep --color=au
to sshd
```

```
john@lab:~$ top -p 1267
top - 21:16:33 up 59 min,  2 users,  load average: 0.00, 0.02, 0.05
任务:      total,      running,      sleeping,      stopped,      zombie
%Cpu(s):    us,        sy,        ni,        id,        wa,        hi,        si,        st
MiB Mem :      total,      free,      used,      buff/cache
MiB Swap:      total,      free,      used.      avail Mem

 进程 USER      PR    NI     VIRT     RES     SHR  %CPU  %MEM    TIME+  COMMAND
 1267 root        20     0    12028    6816   5988 S    0.0   0.2    0:00.00  sshd
```



查看进程树命令pstree

以树状图形式显示进程信息，进程之前的依赖关系，哪个是父进程，哪个是子进程都可以通过pstree进行查看。

```
john@lab:~$ pstree
systemd--ManagementAgent---6*[{ManagementAgent}]
      |--ModemManager---2*[{ModemManager}]
      |--NetworkManager---2*[{NetworkManager}]
      |--VGAuthService
      |--accounts-daemon---2*[{accounts-daemon}]
      |--acpid
      |--avahi-daemon---avahi-daemon
      |--bluetoothd
      |--boltd---2*[{boltd}]
      |--colord---2*[{colord}]
```

```
john@lab:~$ pstree -p
systemd(1)---ManagementAgent(1729)---{ManagementAgent}(1742)
      |---{ManagementAgent}(1743)
      |---{ManagementAgent}(1744)
      |---{ManagementAgent}(1745)
      |---{ManagementAgent}(1746)
      |---{ManagementAgent}(1747)
      |---ModemManager(999)---{ModemManager}(1118)
      |---{ModemManager}(1127)
      |---NetworkManager(989)---{NetworkManager}(1194)
      |---{NetworkManager}(1195)
```



列出进程打开文件信息的命令lsof

lsof命令：

- 1) 用于查看进程打开的文件，打开文件的进程，进程打开的端口(TCP、UDP)。
- 2) **lsof** 需要访问核心内存和各种文件，所以需要root用户执行。

lsof常见命令参数：

<code>lsof abc.txt</code>	显示开启 <u>文件abc.txt</u> 的进程
<code>lsof -c bcd</code>	显示 <u>进程bcd</u> 现在打开的文件
<code>lsof -c -p 1234</code>	列出 <u>进程号为1234</u> 的进程所打开的文件
<code>lsof -g gid</code>	显示归属gid的进程情况
<code>lsof +d /usr/local/</code>	显示目录下被进程开启的文件
<code>lsof +D /usr/local/</code>	同上，但是会搜索目录下的目录，时间较长
<code>lsof -d 4</code>	显示使用fd为4的进程



列出进程打开文件信息的命令lsof

- 【实例】查看与指定文件相关的进程。

```
john@lab:~$ lsof /bin/bash
COMMAND  PID  USER  FD   TYPE    DEVICE  SIZE/OFF      NODE NAME
bash     2868 john   txt    REG     8,17    1166912  1048685 /usr/bin/bash
bash     4985 john   txt    REG     8,17    1166912  1048685 /usr/bin/bash
bash     5084 john   txt    REG     8,17    1166912  1048685 /usr/bin/bash
john@lab:~$
```

COMMAND: 进程的名称

PID: 进程标识符

PPID: 父进程标识符（需要指定-R参数）

USER: 进程所有者

FD: 文件描述符，应用程序通过文件描述符识别该文件。如cwd、txt等

TYPE: 文件类型，如DIR、REG等，常见的文件类型

DEVICE: 指定磁盘的名称

SIZE: 文件的大小

NODE: 索引节点（文件在磁盘上的标识）

NAME: 打开文件的确切名称



列出进程打开文件信息的命令lsof

- 【实例】列出进程调用或打开的所有文件。

```
john@lab:~$ ps aux|grep sshd
root      1267  0.0  0.2 12028 6816 ?        Ss   20:17   0:00 /usr/sbin/sshhd
-D
root      2753  0.0  0.2 12960 8164 ?        Ss   20:20   0:00 sshd: john [priv]
john      2867  0.0  0.2 13272 6456 ?        S    20:20   0:01 sshd: john@pts/0
john      5615  0.0  0.0 17672  848 pts/0    S+   21:38   0:00 grep --color=auto sshd
```

```
john@lab:~$ sudo lsof -p 1267
lsof: WARNING: can't stat() fuse.gvfsd-fuse file system /run/user/1000/gvfs
Output information may be incomplete.
lsof: WARNING: can't stat() fuse file system /run/user/1000/doc
Output information may be incomplete.
COMMAND  PID  USER  FD  TYPE  DEVICE  SIZE/OFF      NODE NAME
sshhd    1267 root   cwd   DIR    8,17    4096         2 /
sshhd    1267 root   rtd   DIR    8,17    4096         2 /
sshhd    1267 root   txt   REG    8,17   843304 1049358 /usr/sbin/sshhd
sshhd    1267 root   mem   REG    8,17    51832 1058367 /usr/lib/x86_64-linux-gnu/libnss_files-2.30.so
```



第7章 进程管理

1. Linux进程概述
2. 进程状态监测
3. 进程状态控制
4. 进程启动与作业控制



进程状态控制

```
kill [-s sigspec | -n signum | -sigspec] pid | jobspec ...  
kill -l [sigspec]
```

选项—

- s signal:** 指定要发送的信号，默认为 **TERM** 信号
- l:** 列出所有可用的信号
- p pid:** 指定要发送信号的进程 ID
- u user:** 指定要发送信号的用户名称
- n sig** 信号名称对应的数字。
- L** 等价于 **-l** 选项。

参数

pid: 进程ID

jobspec: 作业标识符



向进程发送信号的命令kill

```
# kill -l
```

- 1) SIGHUP
- 2) SIGINT
- 3) SIGQUIT
- 4) SIGILL
- 5) SIGTRAP
- 6) SIGABRT
- 7) SIGBUS
- 8) SIGFPE
- 9) SIGKILL
- 10) SIGUSR1
- 11) SIGSEGV
- 12) SIGUSR2
- 13) SIGPIPE
- 14) SIGALRM
- 15) SIGTERM
- 16) SIGSTKFLT
- 17) SIGCHLD
- 18) SIGCONT
- 19) SIGSTOP
- 20) SIGTSTP
- 21) SIGTTIN
- 22) SIGTTOU
- 23) SIGURG
- 24) SIG
- 26) SIGVTALRM
- 27) SIGPROF
- 28) SIGWINCH
- 29)
- 31) SIGSYS
- 34) SIGRTMIN
- 35) SIGRTMIN+1
- 36)
- 38) SIGRTMIN+4
- 39) SIGRTMIN+5
- 40) SIGRTMIN+6
- 43) SIGRTMIN+9
- 44) SIGRTMIN+10
- 45) SIGRTMIN+1
- 48) SIGRTMIN+14
- 49) SIGRTMIN+15
- 50) SIGRTMAX-
- 53) SIGRTMAX-11
- 54) SIGRTMAX-10
- 55) SIGRTMAX-
- 58) SIGRTMAX-6
- 59) SIGRTMAX-5
- 60) SIGRTMAX-4
- 63) SIGRTMAX-1
- 64) SIGRTMAX

HUP	1	终端挂断
INT	2	中断（同 Ctrl + C）
QUIT	3	退出（同 Ctrl + \）
KILL	9	强制终止
TERM	15	终止
CONT	18	继续（与STOP相反，fg/bg命令）
STOP	19	暂停（同 Ctrl + Z）



进程状态控制

1. 运行一个后台任务

```
# sleep 10000 &
```

查看后台运行任务

```
# jobs -l
```

2. 发送停止信号

```
# kill -s STOP XXXXXX
```

发送19信号，停止进程。

```
# kill -n 19 XXXXXX
```

3. 发送继续信号

```
# kill -s CONT XXXXXX
```

```
# jobs -l
```



【实例】使用ps命令查找并杀死进程。

第一步：

```
zp@lab:~$ ps -l
F S    UID      PID      PPID    C  PRI   NI ADDR SZ WCHAN  TTY          TIME CMD
0 S    1000     13442    13441   0   80    0  -   4984 do_wai pts/0        00:00:00 bash
0 T    1000     23131    13442   0   95   15  -   8275 do_sig pts/0        00:00:00 vi
0 T    1000     23139    13442   0   95   15  -   8275 do_sig pts/0        00:00:00 vi
0 T    1000     23155    13442   0   95   15  -   8275 do_sig pts/0        00:00:00 vi
0 R    1000     23274    13442   0   80    0  -   5009 -      pts/0        00:00:00 ps
```

第一步：

```
zp@lab:~$ kill -KILL 23131
zp@lab:~$ ps -l
F S    UID      PID      PPID    C  PRI   NI ADDR SZ WCHAN  TTY          TIME CMD
0 S    1000     13442    13441   0   80    0  -   4984 do_wai pts/0        00:00:00 bash
0 T    1000     23139    13442   0   95   15  -   8275 do_sig pts/0        00:00:00 vi
0 T    1000     23155    13442   0   95   15  -   8275 do_sig pts/0        00:00:00 vi
0 R    1000     23284    13442   0   80    0  -   5009 -      pts/0        00:00:00 ps
[7] 已杀死      nice --adjustment=15 vi
```

```
zp@lab:~$ ps -l
F S    UID      PID      PPID    C  PRI   NI ADDR SZ WCHAN  TTY          TIME CMD
0 S    1000     13442    13441   0   80    0  -   4984 do_wai pts/0        00:00:00 bash
0 T    1000     23139    13442   0   95   15  -   8275 do_sig pts/0        00:00:00 vi
0 T    1000     23155    13442   0   95   15  -   8275 do_sig pts/0        00:00:00 vi
0 R    1000     23284    13442   0   80    0  -   5009 -      pts/0        00:00:00 ps
[7] 已杀死      nice --adjustment=15 vi
zp@lab:~$
zp@lab:~$ kill -9 23139
zp@lab:~$ ps -l
F S    UID      PID      PPID    C  PRI   NI ADDR SZ WCHAN  TTY          TIME CMD
0 S    1000     13442    13441   0   80    0  -   4984 do_wai pts/0        00:00:00 bash
0 T    1000     23155    13442   0   95   15  -   8275 do_sig pts/0        00:00:00 vi
0 R    1000     23305    13442   0   80    0  -   5009 -      pts/0        00:00:00 ps
[8] 已杀死      nice -n 15 vi
```



通过名字杀死进程的命令killall

- 【实例】杀死某一类进程

```
zp@lab:~$ nice -n 15 vi&
[11] 23664
zp@lab:~$ nice -n 15 vi&
[12] 23665

[11]+ 已停止                  nice -n 15 vi
zp@lab:~$ nice -n 15 vi&
[13] 23672

[12]+ 已停止                  nice -n 15 vi
zp@lab:~$ ps -l
F S  UID      PID      PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
0 S   1000    13442    13441  0  80   0  -  4984 do_wai pts/0        00:00:00 bash
0 T   1000    23664    13442  0  95  15  -  8278 do_sig pts/0        00:00:00 vi
0 T   1000    23665    13442  0  95  15  -  8278 do_sig pts/0        00:00:00 vi
0 T   1000    23672    13442  0  95  15  -  8278 do_sig pts/0        00:00:00 vi
0 R   1000    23673    13442  0  80   0  -  5009 -      pts/0        00:00:00 ps

[13]+ 已停止                  nice -n 15 vi
zp@lab:~$ killall -9 vi
[11] 已杀死                  nice -n 15 vi
[12]- 已杀死                  nice -n 15 vi
zp@lab:~$ ps -l
F S  UID      PID      PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
0 S   1000    13442    13441  0  80   0  -  4984 do_wai pts/0        00:00:00 bash
0 R   1000    23682    13442  0  80   0  -  5009 -      pts/0        00:00:00 ps
[13]+ 已杀死                  nice -n 15 vi
```

killall命令用于结束指定名字的进程





【实例】终止指定用户的所有进程

```
zp@lab:~$ ps -ef |grep zp
zp      1657      1  0 19:17 ?          00:00:00 /lib/systemd/systemd --user
zp      1664     1657  0 19:17 ?          00:00:01 [pulseaudio] <defunct>
zp      1674     1657  0 19:17 ?          00:00:00 [dbus-daemon] <defunct>
zp      1703     1649  0 19:17 tty2      00:00:00 /usr/lib/gdm3/gdm-x-session
--run-script env GNOME_SHELL_SESSION_MODE=ubuntu /usr/bin/gnome-session --system
d --session=ubuntu
zp      1764     1703  0 19:17 tty2      00:00:00 /usr/libexec/gnome-session-b
inary --systemd --systemd --session=ubuntu
zp      1840     1764  0 19:17 ?          00:00:00 [ssh-agent] <defunct>
zp      1873     1657  0 19:17 ?          00:00:00 [ibus-portal] <defunct>
zp      1880     1657  0 19:17 ?          00:00:00 [at-spi-bus-laun] <defunct>
```

```
zp      24014    23922  0 21:11 ?          00:00:00 sshd: zp@pts/0
zp      24015    24014  0 21:11 pts/0      00:00:00 -bash
zp      24033    24015  0 21:11 pts/0      00:00:00 ps -ef
zp      24034    24015  0 21:11 pts/0      00:00:00 grep --color=auto zp
zp@lab:~$
zp@lab:~$ killall -u zp
```

killall命令用
于结束指定用
户的所有进程





进程状态控制

• 注意事项:

1) 终止一个前台进程可以使用Ctrl+C键，但是对于一个后台进程就须用kill命令来终止。

2) 要撤销所有的后台作业，可以输入kill 0。使用kill 0来终止所有由当前shell启动的进程，是个有效的方法。

3) 当用kill向这些进程发送信号时，必须是这些进程的主人。如果试图撤销一个没有权限撤销的进程或撤销一个不存在的进程，就会得到一个错误信息。

4) init进程是不可终止的！进程编号为1。



第7章 进程管理

1. Linux进程概述
2. 进程状态监测
3. 进程状态控制
4. 进程启动与作业控制



进程启动与作业控制

- 通常将正在执行的一个或者多个相关进程称为一个作业（job）
 - 作业是用户向计算机提交任务的任务实体。
 - 而进程则是完成用户任务的执行实体，是向系统申请分配资源的基本单位。
 - 一个作业可以包含一个或者多个进程。
 - 用户通过作业控制可以将进程挂起，也可以在需要时将其恢复运行。



第7章 进程管理

什么是作业？

- 1) 就是用户提交给系统的一个任务。
- 2) 若干进程共同完成一个作业。作业可以包含一个或多个进程。

jobs的状态可以是running, stopped, Terminated。

什么是前台和后台：

(一) 前台：是在终端中运行的命令，那么该终端就是进程的控制终端，一旦这个终端关闭，这个进程也随之消失。

(二) 后台：是运行在后台的进程，不受终端控制，它不需要终端的交互。



第7章 进程管理

`jobs` 命令——用来查看当前终端放在后台的进程。

如何进行前后台切换：

1. 将前台切换到后台

先使用`Ctrl+z`把作业暂停，然后用 `bg %jobid`命令把它放到后台运行

2. 将后台切换到前台

`fg %jobid`（或者简单的用`%jobid`）将后台中的命令调至前台继续运行。



第7章 进程管理

如何终止作业：

- (一) 使用`kill %jobid`，终止作业。%%代表最近的作业，%-代表次最近的作业
- (二) 使用`kill pid`，终止进程
- (三) 前台进程或作业的终止：使用`ctrl+c`



使用nohup命令启动脱离终端运行的任务

nohup命令：该命令可以在你退出帐户/关闭终端之后继续运行相应的进程。

- **【实例】nohup命令综合实例。**
 - 本实例中，将一条常见的ping命令，移动到后台，并且脱离终端运行。然后，在重新连接后，通过进程管理命令，找出该后台运行进程，查看其记录信息，并该进程进行管理和维护操作。

```
zp@lab:~$ ping www.ubuntu.com
PING www.ubuntu.com (91.189.88.180) 56(84) bytes of data.
64 bytes from cactuar.canonical.com (91.189.88.180): icmp_seq=3 ttl=128 time=308 ms
64 bytes from cactuar.canonical.com (91.189.88.180): icmp_seq=4 ttl=128 time=289 ms
64 bytes from cactuar.canonical.com (91.189.88.180): icmp_seq=5 ttl=128 time=288 ms
64 bytes from cactuar.canonical.com (91.189.88.180): icmp_seq=7 ttl=128 time=435 ms
64 bytes from cactuar.canonical.com (91.189.88.180): icmp_seq=8 ttl=128 time=291 ms
```



- 需要注意，再次连接后，jobs并不能看到该进程信息。可以通过ps找出该进程，然后通过kill指令结束该进程。

```
zp@lab:~$ nohup ping www.ubuntu.com &
[1] 28438
zp@lab:~$ nohup: 忽略输入并把输出追加到'nohup.out'

zp@lab:~$
zp@lab:~$ ls nohup.out
nohup.out
zp@lab:~$
zp@lab:~$ jobs
[1]+  运行中                  nohup ping www.ubuntu.com &
zp@lab:~$
```

```
zp@lab:~$ tail nohup.out -f
64 bytes from elvira.canonical.com (91.189.91.44): icmp_seq=257 ttl=128 time=477 ms
64 bytes from elvira.canonical.com (91.189.91.44): icmp_seq=258 ttl=128 time=265 ms
64 bytes from elvira.canonical.com (91.189.91.44): icmp_seq=259 ttl=128 time=267 ms
64 bytes from elvira.canonical.com (91.189.91.44): icmp_seq=260 ttl=128 time=263 ms
64 bytes from elvira.canonical.com (91.189.91.44): icmp_seq=262 ttl=128 time=567 ms
64 bytes from elvira.canonical.com (91.189.91.44): icmp_seq=267 ttl=128 time=600 ms
64 bytes from elvira.canonical.com (91.189.91.44): icmp_seq=268 ttl=128 time=265 ms
64 bytes from elvira.canonical.com (91.189.91.44): icmp_seq=271 ttl=128 time=404 ms
64 bytes from elvira.canonical.com (91.189.91.44): icmp_seq=273 ttl=128 time=308 ms
```

```
zp@lab:~$ jobs
zp@lab:~$
zp@lab:~$
zp@lab:~$ ps -ef |grep ping
zp      28438      1  0 22:26 ?        00:00:00 ping www.ubuntu.com
zp      29065    28781  0 22:33 pts/0    00:00:00 grep --color=auto ping
zp@lab:~$ kill -9 28438
zp@lab:~$ ps -ef |grep ping
zp      29104    28781  0 22:33 pts/0    00:00:00 grep --color=auto ping
```